



SUBJECT : PROGRAMMING IN JAVA

SUBJECT CODE : CSI26W1

LEARNING UNIT: 4

PROGRAMMING EXERCICES

Task 1:

- a) Create a class named `Dog` that contains data fields for the name, color, and birth year. Include get and set methods for these fields.
- b) Create a subclass of `Dog` named `RacingDog`, which contains an additional field that holds the number of races in which the `Dog` has competed and additional methods to get and set the new field.
- c) Write an application that prompts the user for appropriate field values and demonstrates using objects of each class, `Dog` and `RacingDog`. Save the file as **DemoDog.java**.

Task 2:

- a) Create a `GucciShirt` class for Dragon City Company. Fields include an order number, size, color, and price. Create set methods for the order number, size, and color, and create get methods for all four fields. The price is determined by the size: R299.99 for XXL or XXXL, and R199.99 for all other sizes.
- b) Create a subclass named `PersonalizedShirt` that descends from `GucciShirt` and includes a field to hold the slogan requested for the shirt. Include get and set methods for this field.
- c) Write an application that prompts a user for values, creates two objects of each class, and demonstrates that all the methods work correctly. Save the file as **GucciDemo.java**

Task 3:

- a) Create a class named `ChessGame` that contains data fields for two player names. Include a two-dimensional array that can hold the scores for each of two players across a number of game rounds. The `ChessGame` class includes a `final int` that

represents the number of rounds in a standard chess match—nine. The constructor initializes the two-dimensional array to dummy values for each player for the number of rounds. Create set methods for the player names and the individual round scores. The `setScore()` method requires parameters for the player, round, and score for that round. Create get methods for the player names and round scores.

- b) Create two subclasses from `ChessGame`: `ProfessionalGame` and `AmateurGame`. Professional chess games have seven rounds and Amateur chess games have five rounds. Override the parent class's `final int` to represent the correct number of rounds in each subclass. The constructor for each class should initialize the scores to a default value for the appropriate number of rounds.
- c) Write an application that allows the user to select a game type and enter two player names. Then, for each round, allow the user to enter a score for each player (e.g., 1 for win, 0.5 for draw, 0 for loss), and display the running scores so far. After all rounds are complete, display a message indicating which player won or if the result was a tie. Save the file as **DemoChessGame.java**.

Task 4:

- a) Create an abstract class named `Book`. Include a `String` field for the `Book`'s title and a `double` field for the book's price. Within the class, include a constructor that requires the book title, and add two get methods—one that returns the title and one that returns the price. Include an abstract method named `setPrice()`.
- b) Create two child classes of `Book`: `Fiction` and `NonFiction`. Each must include a `setPrice()` method that sets the price for all `Fiction` Books to R240.99 and for all `NonFiction` Books to R370.99. Write a constructor for each subclass, and include a call to `setPrice()` within each.
- c) Write an application demonstrating that you can create both a `Fiction` and `NonFiction` `Book`, prompt the user for values, and display their fields. Save the file as **BookDemo.java**.
- d) Write an application named `BookArray` in which you create an array that holds 10 `Books`, some `Fiction` and some `NonFiction`. Using a `for` loop, prompt the user for a title and whether the `Book` is fiction or nonfiction (*F* or *N*). Create the appropriate book type and store it in the array. If the user does not select *F* or *N*, reprompt the user. In another `for` loop, display details about all 10 `Books`. Save the file as **BookArray.java**.

Task 5:

- a) Create an abstract `Division` class with fields for a company's division name and account number, and an abstract `display()` method. Use a constructor in the superclass that requires values for both fields.
- b) Create two subclasses named `InternationalDivision` and `DomesticDivision`. The `InternationalDivision` subclass includes a field for the country in which the division is located and a field for the language spoken; its constructor requires both. The `DomesticDivision` subclass includes a field for the state in which the division is located; a value for this field is required by the constructor.
- c) Write an application named `UseDivision` that creates a `Division` reference. Ask the user whether a `DomesticDivision` or `InternationalDivision` object should be created. Prompt for appropriate values and assign a new subclass object to the `Division` reference. Then display the details of the created object. Save the file as **`UseDivision.java`**.

~~~~~